Vehicle Normalization Framework

1st Diego Quezada
Departamento de Informática
Universidad Técnica Federico Santa María
Valparaíso, Chile
diego.quezadac@sansano.usm.cl

2nd Carlos Valle
Escuela de Ingeniería Informática
PUCV
Valparaíso, Chile
carlos.valle@pucv.cl

3rd Ricardo Ñanculef
Departamento de Informática
Universidad Técnica Federico Santa María
Valparaíso, Chile
jnancu@inf.utfsm.cl

***Abstract*—**

***Index Terms*—LLM, RAG, Information Extraction, Information Retrieval, Entity Resolution**

I. Introduction

The global second-hand car market, valued at approximately USD 1.90 trillion in 2024, is projected to grow at a compound annual growth rate (CAGR) of 9.4% from 2022 to 2027 [1]. Key factors driving this growth include rising new vehicle prices, enhanced vehicle financing options, and advancements in online platforms [1], [2], which facilitate access to inventory and information impacting consumer trust, such as online reviews and ratings [3].

The expansion of online platforms has produced a proliferation of heterogeneous vehicle datasets across automotive markets. Without a shared standard for representing vehicles, descriptions of the same real-world entity diverge along three axes: **naming inconsistency**, where the same concept appears under different strings (e.g., "mercedes", "mercedes-benz", "mercedes benz"); **granularity inconsistency**, where the same attribute is expressed at different levels of detail (e.g., "automatic" vs. "7-speed DCT"); and **scope inconsistency**, where a single field conflates values that belong to distinct real-world dimensions (e.g., a *fuel type* column containing both "petrol" and "phev"). Left unaddressed, these inconsistencies propagates through downstream machine learning (ML) pipelines, undermining feature extraction, similarity computation, and ultimately model performance.

Several standardization efforts have attempted to formalize vehicle data representation, yet none has produced a solution applicable to open, cross-market datasets. The Vehicle Identification Number (VIN), standardized under ISO 3779 [4], has proven effective within closed industry systems and dealership networks, but its decoding tables are proprietary, vary across manufacturers, and VIN data is rarely present in open datasets. The NHTSA vPIC database [5] decodes VINs into approximately 130 attributes but is restricted to US-market vehicles and excludes condition, usage history, and equipment. The ACES/PIES standards [6] provide structured fitment data for aftermarket parts in North America, not whole-vehicle attributes. The W3C Automotive Ontology and its `schema.org` extension [7] define a minimal set of vehicle properties for web markup, but accept free-form text for key fields and omit trim, equipment, and condition entirely. COVESA's Vehicle Signal Specification (VSS) [8] standardizes in-vehicle telemetry signals rather than vehicle-as-product attributes, and its own gap analysis acknowledges persistent interoperability issues across OEMs [9]. Together, these efforts address narrow, well-scoped problems within specific industry contexts, leaving the challenge of normalizing heterogeneous vehicle listings from open source datasets largely unresolved. This problem has been repeatedly noted in the vehicle price prediction literature [10]–[12], where studies report that extensive ad-hoc preprocessing is required for each new dataset and call for standardized representation protocols.

In this work, we propose a general vehicle normalization framework that leverages recent advances in large language models (LLMs) applied to information extraction (IE), information retrieval (IR), and entity resolution (ER) to map any raw vehicle description to a normalized, domain-consistent representation suitable for downstream machine learning (ML) tasks. Our framework is open-source and extensible, allowing practitioners to define custom output schemas tailored to their specific attribute requirements, and is designed to serve as a shared baseline for future research on ML applications in the vehicle domain.

II. Related Work

A. Vehicle Data Preprocessing in ML

Every ML study on vehicle data confronts the fact that each dataset uses its own representation: different columns, naming conventions, languages, abbreviations, and attribute domains. In practice, each study builds a tailored preprocessing pipeline from scratch, and none of the resulting transformations transfers to a new dataset or market.

Bergmann and Feuerriegel [13] provide the most detailed example of this pattern. Working with 92,239 used car sales from a Swiss retailer, they faced approximately 50,000 unique equipment identifiers represented as short German-language text descriptions (e.g., *"Navigationsfunktion 'Discover media' fuer Radiosystem 'Composition media' "*). Their preprocessing pipeline consisted of three steps: first, they manually selected 18 target equipment categories (e.g., navigation system, alloy rims, park assist) based on the search filters of a Swiss online platform, validated by 51 car dealers; second, they mapped the 50,000 textual descriptions onto these 18 categories using

Identify applicable funding agency here. If none, delete this.

a keyword-based search combined with regular expressions; third, they applied one-hot encoding to produce 18 binary predictors. This pipeline improved prediction performance (MAE) by 3.27%. Crucially, the authors describe their approach as "tailored" to their partner company's data and market. The keyword rules are language-specific (German), the 18 categories reflect Swiss consumer preferences, and the mapping logic would need to be rebuilt for any new market, language, or equipment taxonomy.

Fayyaz et al. [11] expose how raw vehicle listings resist direct ML application. Working with real-world listing data, they found that transmission entries varied widely (e.g., "6-Speed A/T", "8-Speed PDK", "CVT-F"), mileage was stored as strings (e.g., "10,000 mi."), and engine specifications were embedded in free text (e.g., "2.0L 255HP") requiring regex extraction. They also standardized color names from manufacturer branding (e.g., "Midnight Black" mapped to "Black"). After systematic feature engineering, Linear Regression R^2 improved from 0.03 to 0.80, yet the authors acknowledge that their pipeline is dataset-specific. Dutulescu et al. [10] confirm that this problem extends across markets: training deep learning models on a German dataset (300,000+ entries) and evaluating on a Romanian dataset (15,000+ entries), they found that car model embeddings failed for infrequent models and explicitly called for "a method for dataset standardization across different car markets." Madhusudhanan et al. [12] faced similar challenges at larger scale, handling 56 categorical features (make, model, trim, equipment, market region) across approximately 2 million records, where state-of-the-art tabular models could not even process the dataset due to the high cardinality of unstandardized vehicle attributes.

These studies collectively demonstrate that vehicle data preprocessing remains ad-hoc, non-transferable, and a significant engineering bottleneck. Our framework addresses this gap by providing a reusable normalization pipeline that is agnostic to source schema, language, and market.

B. Information Extraction

IE from product descriptions has evolved from task-specific sequence taggers to general-purpose LLM-based extractors. OpenTag [14] formalized product attribute value extraction (AVE) as sequence tagging with a BiLSTM-CRF architecture, achieving 83% F1 with only 150 annotated samples. Wang et al. [15] reformulated AVE as question answering, where each attribute becomes a query and the model identifies answer spans using a shared BERT encoder.

The most relevant line of work for our framework comes from Brinkmann et al. In an initial study [16], they showed that ChatGPT achieves performance comparable to fine-tuned pre-trained language models (PLMs) for product information extraction while requiring far less training data: a single few-shot example boosted open extraction F1 by approximately 33%. Their follow-up, ExtractGPT [17], evaluated GPT-3.5, GPT-4, and Llama-3-70B on attribute-value extraction from unstructured product descriptions, with GPT-4 reaching 85% F1 and outperforming BERT-based baselines. Most directly relevant to our work, Brinkmann et al. [18] extended AVE to include *normalization*, exploring GPT-3.5 and GPT-4 for both extracting and normalizing attribute values to a unified scale. On the WDC-PAVE benchmark, GPT-4 achieved 91% F1 with 10 demonstrations, and the authors found LLMs to be especially strong at name expansion and string wrangling for normalization, which is precisely the capability our framework exploits for vehicle attribute extraction.

Beyond product-specific work, GoLLIE [19] demonstrated that encoding annotation guidelines as Python class definitions with docstrings enables high-quality zero-shot IE, outperforming GPT-3.5 on NER tasks. This finding is directly relevant to our approach, where the vehicle attribute schema serves an analogous role as a structured extraction guide.

C. Information Retrieval

The RAG paradigm, introduced by Lewis et al. [20], combines parametric (model weights) and non-parametric (retrieved documents) knowledge by coupling a seq2seq model with a dense vector index. For the retrieval component specifically, Karpukhin et al. [21] established that learned dual-encoder embeddings (Dense Passage Retrieval) outperform BM25 by 9–19% absolute in top-20 retrieval accuracy. Izacard et al. [22] showed that fully unsupervised dense retrieval via contrastive learning matches BM25 without requiring labeled query-passage pairs, which is particularly relevant when labeled vehicle matching data is scarce.

For efficient nearest-neighbor search over embedding spaces, the FAISS library [23], [24] provides GPU-optimized algorithms supporting brute-force, inverted file (IVF), product quantization (PQ), and hierarchical navigable small world (HNSW) indexing. On the embedding side, Sentence-BERT [25] established the paradigm of pre-computed sentence embeddings for efficient similarity search, while recent models such as E5 [26] and BGE [27] have achieved state-of-the-art results on retrieval benchmarks. Our framework uses HNSW indexes built over sentence embeddings to retrieve candidate values for open-domain vehicle attributes.

D. Entity Resolution

ER determines whether two surface forms refer to the same real-world entity (e.g., whether "mercedes" and "mercedes-benz" denote the same brand). Pre-LLM approaches relied on fine-tuning PLMs: Ditto [28] fine-tuned BERT and RoBERTa for entity matching with domain knowledge injection and data augmentation, achieving up to 29% F1 improvement over prior methods. Narayan et al. [29] were the first to cast entity matching as a prompting task for GPT-3, showing that foundation models achieve competitive performance without task-specific fine-tuning.

The most systematic investigation of LLM-based ER comes from the Mannheim group. Peeters and Bizer [30] conducted the first evaluation of ChatGPT for entity matching, finding that it achieves 82.35% F1 in zero-shot on the WDC Products benchmark, competitive with RoBERTa fine-tuned on 2,000 labeled examples. They explored prompt engineering

strategies including similarity-based demonstration selection and matching rule provision. In their comprehensive follow-up, MatchGPT [31], they evaluated GPT-4, GPT-4o, and Llama across product and bibliographic datasets. GPT-4 in zero-shot outperformed fine-tuned PLMs by 40–68% F1 for previously unseen entities, a setting directly analogous to vehicle normalization where new brands, models, and trims continuously appear. They further investigated prompt design, demonstration selection, and matching rule generation, providing a methodological foundation for LLM-based matching in specialized domains. Steiner et al. [32] complemented this work by exploring fine-tuning LLMs for entity matching, finding that it improves smaller models but can hurt cross-domain transfer for larger ones.

On the efficiency front, Zhang et al. [33] showed that a fine-tuned GPT-2 model achieves quality within 4.4% F1 of MatchGPT while requiring four orders of magnitude fewer parameters, demonstrating that cost-effective deployment is feasible. Wang et al. [34] compared three LLM matching strategies (binary matching, comparing, and selecting from candidates) and found the selecting strategy most cost-effective.

III. Method

Let the normalized vehicle schema consist of m attributes $A_1, A_2, \ldots, A_m$, where each attribute A_j is associated with a domain $\mathcal{V}_j$ of valid values. Attribute domains are partitioned into two types:

$$\mathcal{V}_j = \begin{cases} \mathcal{V}_j^{\text{closed}} & \text{if } j \in \mathcal{I}_{\text{closed}}, \\ \mathcal{V}_j^{\text{open}} & \text{if } j \in \mathcal{I}_{\text{open}}. \end{cases} \tag{1}$$

A *closed-domain* attribute has a domain that is fully specified in advance and does not change during normalization. This includes both finite categorical sets (e.g., fuel type $\in$ {petrol, diesel, lpg, hydrogen}) and numerical ranges (e.g., mileage $\in \mathbb{R}^+$). An *open-domain* attribute (e.g., brand, model) has a domain that grows as the system encounters previously unseen entities.

The normalized vehicle space is the Cartesian product of all attribute domains:

$$\tilde{\mathcal{X}} = \prod_{j=1}^{m} \mathcal{V}_j. \tag{2}$$

Vehicle normalization is formulated as a mapping

$$\phi : \Sigma^+ \to \tilde{\mathcal{X}}, \tag{3}$$

that takes a serialized vehicle representation $s = \text{Serialize}(\mathbf{x}) \in \Sigma^+$, where $\mathbf{x}$ is the raw input record and Σ denotes the character alphabet, and produces $\phi(s) = \tilde{\mathbf{x}} \in \tilde{\mathcal{X}}$. For tabular datasets, Serialize concatenates column names with their values (e.g., *col1: val1, col2: val2, …*); for free-text descriptions, the input is used directly.

a) Knowledge catalog.: For each open-domain attribute A_j with $j \in \mathcal{I}_{\text{open}}$, we maintain a knowledge catalog:

$$\mathcal{K}_j = \{(c_1, \mathbf{e}_1), \ldots, (c_{|\mathcal{K}_j|}, \mathbf{e}_{|\mathcal{K}_j|})\}, \tag{4}$$

where each pair $(c_i, \mathbf{e}_i)$ consists of a canonical entity value $c_i \in \mathcal{V}_j^{\text{open}}$ and its embedding $\mathbf{e}_i = \text{Encode}(c_i) \in \mathbb{R}^d$. Any text encoder may serve as the encoding function; in this study we use OpenAI's *text-embedding-3-small*.

b) Information extraction.: Given a serialized vehicle description s, a single LLM call extracts candidate values for all attributes simultaneously:

$$\hat{\mathbf{x}} = (\hat{a}_1, \ldots, \hat{a}_m) = \text{LLM}(\text{prompt}_{\text{ext}}, s). \tag{5}$$

For closed-domain attributes, the extracted value is used directly as $\tilde{a}_j = \hat{a}_j$, since the prompt constrains the LLM output to $\mathcal{V}_j^{\text{closed}}$. For open-domain attributes, the candidate $\hat{a}_j$ is passed to the retrieval and resolution steps described below.

c) Information retrieval.: For each open-domain attribute A_j, the extracted candidate is embedded as $\mathbf{e}_j = \text{Encode}(\hat{a}_j)$, and the top-$k$ most similar canonical entities are retrieved from catalog $\mathcal{K}_j$:

$$\mathcal{R}_j = \text{top-k}_{(c_i, \mathbf{e}_i) \in \mathcal{K}_j} \ \text{sim}(\mathbf{e}_j, \mathbf{e}_i), \quad \text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \, \|\mathbf{v}\|_2}. \tag{6}$$

The candidates in $\mathcal{R}_j$ are ranked in decreasing order of similarity.

d) Entity resolution.: Given the extracted attributes $\hat{\mathbf{x}}$ and the ranked candidates $\mathcal{R}_j = (c_1, c_2, \ldots, c_k)$ for each open-domain attribute A_j, we compare two serialized vehicle descriptions. Let

$$\hat{s} = \text{Serialize}(\hat{\mathbf{x}}) \tag{7}$$

be the serialization of the extracted attributes. For each candidate $c_i \in \mathcal{R}_j$, we construct a variant in which the value of attribute A_j is replaced by c_i:

$$\hat{s}_{j \to c_i} = \text{Serialize}(\hat{a}_1, \ldots, c_i, \ldots, \hat{a}_m). \tag{8}$$

The LLM then evaluates whether both descriptions refer to the same vehicle:

$$M(\hat{s}, \hat{s}_{j \to c_i}) = \text{LLM}(\text{prompt}_{\text{res}}, \hat{s}, \hat{s}_{j \to c_i}) \in \{0, 1\}. \tag{9}$$

The iteration proceeds in similarity-ranked order and stops at the first match; let $i^* = \min\{i : M(\hat{s}, \hat{s}_{j \to c_i}) = 1\}$ denote the rank of that match. The normalized value is:

$$\tilde{a}_j = \begin{cases} c_{i^*} & \text{if a match exists,} \\ \hat{a}_j & \text{otherwise (novel entity).} \end{cases} \tag{10}$$

In the novel-entity case, the catalog is expanded: $\mathcal{K}_j \leftarrow \mathcal{K}_j \cup \{(\hat{a}_j, \text{Encode}(\hat{a}_j))\}$.

Algorithm 1 summarizes the complete pipeline.

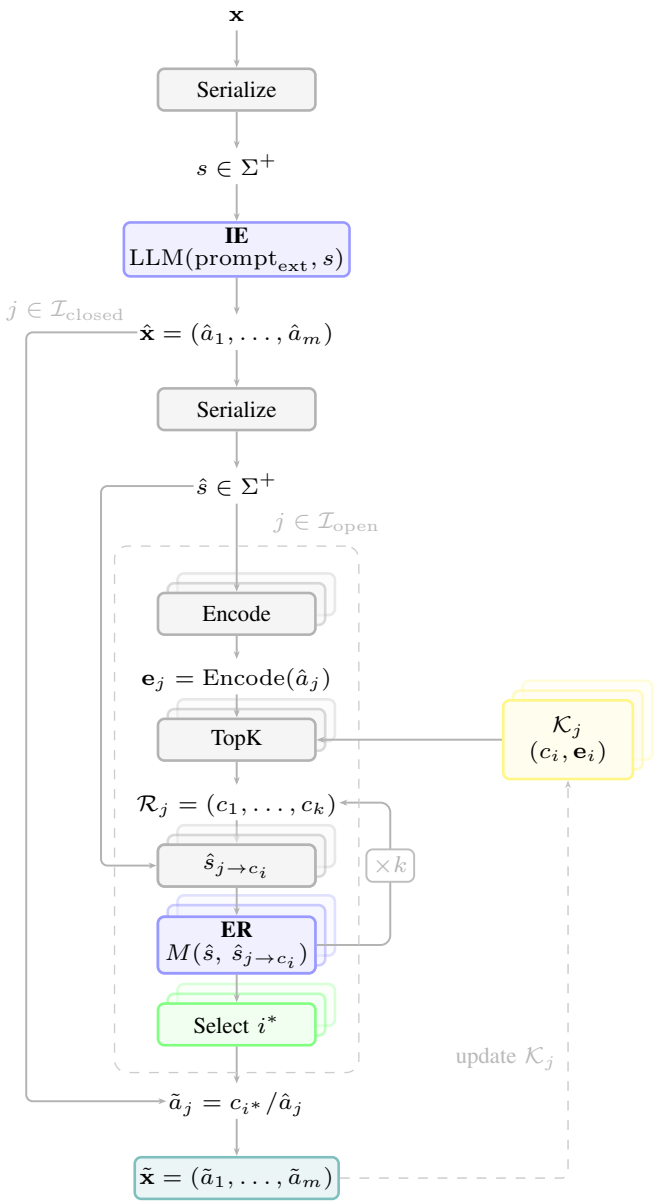

Fig. 1. Vehicle normalization pipeline. The raw record $\mathbf{x}$ is serialized and passed to the LLM for attribute extraction (IE). For each open-domain attribute (outer loop, dashed box; stacked blocks indicate per-attribute instances), the candidate $\hat{a}_j$ is embedded and matched against its catalog $\mathcal{K}_j$ via HNSW, retrieving top-k candidates $\mathcal{R}_j$. For each candidate c_i ($\times k$ return arrow), the pipeline substitutes c_i into the serialized description and evaluates an ER match $M(\hat{s}, \hat{s}_{j \to c_i})$. A selection step returns the first match c_{i^*} or falls back to $\hat{a}_j$. Closed-domain attributes bypass the loop and feed $\hat{a}_j$ directly into the assembly. Novel entities expand the catalog.

Algorithm 1 Vehicle Normalization

Require: Raw vehicle record $\mathbf{x}$, Catalogs $\{\mathcal{K}_j\}_{j \in \mathcal{I}_{\mathrm{open}}}$
Ensure: Normalized representation $\tilde{\mathbf{x}} = (\tilde{a}_1, \ldots, \tilde{a}_m)$

$s \leftarrow \mathrm{Serialize}(\mathbf{x})$
$\hat{\mathbf{x}} \leftarrow \mathrm{LLM}(\mathrm{prompt}_{\mathrm{ext}}, s)$
$\hat{s} \leftarrow \mathrm{Serialize}(\hat{\mathbf{x}})$
for $j \in \mathcal{I}_{\mathrm{closed}}$ **do**
  $\tilde{a}_j \leftarrow \hat{a}_j$
end for
for $j \in \mathcal{I}_{\mathrm{open}}$ **do**
  $\mathbf{e}_j \leftarrow \mathrm{Encode}(\hat{a}_j)$
  $\mathcal{R}_j \leftarrow \mathrm{TopK}_{(c_i, \mathbf{e}_i) \in \mathcal{K}_j} \mathrm{sim}(\mathbf{e}_j, \mathbf{e}_i)$
  $\tilde{a}_j \leftarrow \hat{a}_j$ ▷ default: novel entity
  for $c_i \in \mathcal{R}_j$ in ranked order **do**
    $\hat{s}_{j \to c_i} \leftarrow \mathrm{Serialize}(\hat{a}_1, \ldots, c_i, \ldots, \hat{a}_m)$
    if $M(\hat{s},\ \hat{s}_{j \to c_i}) = 1$ **then**
      $\tilde{a}_j \leftarrow c_i$
      break
    end if
  end for
  if $\tilde{a}_j = \hat{a}_j$ **then**
    $\mathcal{K}_j \leftarrow \mathcal{K}_j \cup \{(\hat{a}_j, \mathrm{Encode}(\hat{a}_j))\}$
  end if
end for
return $\tilde{\mathbf{x}} = (\tilde{a}_1, \ldots, \tilde{a}_m)$

IV. Experiments

A. Datasets

Our experiments draw on three publicly available automotive listing datasets covering distinct geographic markets, schema conventions, and languages, summarized in Table I. AutoScout24 is a European dataset with structured fields and mileage reported in kilometers. Craigslist is a North American dataset characterized by informal free-text descriptions and mileage in miles; its 29,667 nominal model values reflect raw seller input rather than a controlled vocabulary, and are expected to collapse substantially after normalization. MuCars is a Moroccan dataset with several market-specific attributes, including mileage reported as ranges and Fiscal Power as a tax-based engine category rather than a direct power measurement. The heterogeneity across markets, units, and naming conventions makes these three sources well suited for evaluating the robustness of the normalization method.

TABLE I
Dataset Statistics

Dataset	Listings	Brands	Models	Cols	Region
AutoScout24	251,079	47	1,312	15	Europe
Craigslist	426,880	42	29,667	26	North America
MuCars	101,896	81	1,049	15	Morocco

B. Annotation Protocol

Ground truth for IE and ER is produced by a three-model LLM ensemble acting as independent annotators: **GPT-5.4** (OpenAI), **Claude Opus 4.6** (Anthropic), and **Gemini 3.1 Pro** (Google DeepMind). Each annotator is prompted zero-shot with the task instruction and the input example; the full prompts are provided in Appendix A.

Labels are assigned by majority vote: a label is accepted when at least two of the three annotators agree. For IE, this criterion is applied field-by-field; a field for which all three

annotators produce different values is marked ABSTAIN and excluded from evaluation. For ER, the decision is binary (match or no-match), so a majority always exists unless one annotator withholds judgment; such pairs are discarded and replaced by resampling. In both settings we process instances until 100 fully annotated examples are collected, sampling in a stratified manner to ensure coverage of the attribute value distribution.

We report inter-annotator agreement (IAA) as a data quality indicator alongside the primary evaluation metrics: Fleiss' κ [35] over all three annotators and the pairwise raw agreement rate P_A, computed for each of the three annotator pairs and then averaged.

C. Evaluation Setup

Information Extraction. IE is evaluated on each of the three sources independently. For AutoScout24 and MuCars, which contain no free-text field, the input is the serialized structured row (comma-separated *col: value* pairs). For Craigslist, the raw free-text listing description is used directly. We use 100 labeled examples per source, stratified across the distribution of key attribute values (brand, fuel type, transmission type), yielding 300 evaluation instances in total.

Information Retrieval. IR depends on a pre-populated catalog, so we adopt a leave-one-source-out rotation: two sources act as *catalog sources* and the third as the *evaluation source*. The catalog and HNSW indexes are constructed from all rows in the two catalog sources. For each row in the evaluation source, IE is first run to extract a candidate value $\hat{a}_j$ for each open-domain attribute. That candidate is embedded as $\mathbf{e}_j = \text{Encode}(\text{attr}\colon \hat{a}_j)$ and queried against the attribute-specific HNSW index. Success is defined as the correct canonical value, taken from the structured column of the evaluation-source row, appearing among the top-k retrieved candidates. Ground truth is derived directly from structured columns and does not require LLM annotation. We sample 100 queries per (attribute, evaluation source) pair, stratified across the canonical value distribution of that attribute.

Entity Resolution. ER follows the same leave-one-source-out rotation. For each evaluation configuration, we construct a labeled set of candidate pairs drawn entirely from the evaluation source. Positive pairs are formed by sampling surface-form variants of the same canonical entity (e.g., rows where the brand field contains *mercedes* alongside rows containing *mercedes-benz*). Negative pairs include random non-matching entities and hard negatives drawn from adjacent model numbers, parent brand names, and trim levels differing by a single qualifier. We target 100 labeled pairs per (attribute, evaluation source) combination, with positive and negative pairs balanced and stratified to cover diverse surface-form variants and hard negatives. The few-shot resolution prompt used by the pipeline is drawn exclusively from the catalog sources and is distinct from the annotator prompts.

D. Evaluation Metrics

Extraction. For categorical and integer fields we report per-field accuracy as the fraction of examples for which the predicted value exactly matches the annotation. For numeric fields with continuous values (engine displacement, mileage) we apply a $\pm 10\%$ relative tolerance. For the equipment field, which is a set-valued output, we report set-level precision, recall, and F1 score. A macro-average accuracy is reported across all non-set fields.

Retrieval. We report Recall@k for $k \in \{1, 3, 5\}$, defined as the fraction of (query, attribute) pairs for which the correct canonical value appears among the top-k retrieved candidates. Because retrieval feeds directly into the resolution step, a miss at retrieval silently prevents the correct canonical from ever being considered, making recall the primary concern.

Entity Resolution. We report precision, recall, F1 score, and accuracy for the binary match/no-match decision, broken down by attribute. Precision is the primary metric of interest, as a false positive permanently merges two distinct entities in the catalog and is difficult to detect post-hoc.

Inter-annotator agreement. We report Fleiss' κ and the pairwise raw agreement rate P_A for both IE and ER annotation rounds as a measure of label reliability. A $\kappa \geq 0.80$ indicates near-perfect agreement [36]; fields or pairs falling below $\kappa = 0.60$ are flagged and analyzed separately.

E. Results

V. Discussion

VI. Conclusion

Acknowledgment

References

[1] S. Hu, S. X. Zhu, and K. Fu, "The manufacturer's resale strategy for trade-ins," *European Journal of Operational Research*, vol. 323, no. 2, pp. 583–598, 2025.

[2] N. Zacharof, J. Nur, D. Kourtesis, J. Krause, and G. Fontaras, "A review of the used car market in the european union," Luxembourg (Luxembourg), 2025.

[3] Zhang, Minshun, "Decoding consumer behavior in the used car market: A machine learning approach to key decision factors," *ITM Web Conf.*, vol. 70, p. 04033, 2025.

[4] International Organization for Standardization, "Road vehicles – vehicle identification number (vin) – content and structure," ISO 3779:2009, International Organization for Standardization, Geneva, Switzerland, 2009, accessed: December 8, 2025. [Online]. Available: https://www.iso.org/standard/52200.html

[5] National Highway Traffic Safety Administration, "Product information catalog and vehicle listing (vPIC)," 2025, accessed: 2025-04-06. [Online]. Available: https://vpic.nhtsa.dot.gov/

[6] Auto Care Association, "ACES and PIES data standards," 2026, ACES 5.0 and PIES 8.0 released March 2026. [Online]. Available: https://www.autocare.org/data-standards

[7] W3C Automotive Ontology Community Group, "Automotive extension—Schema.org," 2024, accessed: 2025-04-06. [Online]. Available: https://schema.org/docs/automotive.html

[8] COVESA, "Vehicle signal specification (VSS)," 2025, accessed: 2025-04-06. [Online]. Available: https://covesa.global/vehicle-signal-specification/

[9] ——, "Vehicle data model requirements and proposed solutions," 2025, accessed: 2025-04-06. [Online]. Available: https://wiki.covesa.global/display/WIK4/Vehicle+Data+Model+Requirements+and+Proposed+Solutions

[10] A. Dutulescu, A. Catruna, S. Ruseti, D. Iorga, V. Ghita, L.-M. Neagu, and M. Dascalu, "Car price quotes driven by data-comprehensive predictions grounded in deep learning techniques," *Electronics*, vol. 12, no. 14, 2023.
[11] I. Fayyaz, G. G. M. N. Ali, and S. S. Khairunnesa, "Advanced feature engineering and machine learning techniques for high accurate price prediction of heterogeneous pre-own cars," *Vehicles*, vol. 7, no. 3, 2025.
[12] K. Madhusudhanan, G. Behrens, M. Stubbemann, and L. Schmidt-Thieme, "Probsaint: Probabilistic tabular regression for used car pricing," in *2024 IEEE International Conference on Big Data (BigData)*, 2024, pp. 2179–2187.
[13] S. Bergmann and S. Feuerriegel, "Machine learning for predicting used car resale prices using granular vehicle equipment information," *Expert Systems with Applications*, vol. 263, p. 125640, 2025.
[14] G. Zheng, S. Mukherjee, X. L. Dong, and F. Li, "OpenTag: Open attribute value extraction from product profiles," in *Proc. ACM SIGKDD*, 2018, pp. 1049–1058.
[15] Q. Wang, L. Yang, B. Kanagal, S. Sanghai, D. Sivakumar, B. Shu, Z. Yu, and J. Elsas, "Learning to extract attribute value from product via question answering: A multi-task approach," in *Proc. ACM SIGKDD*, 2020, pp. 47–55.
[16] A. Brinkmann, R. Shraga, and C. Bizer, "Product information extraction using ChatGPT," *arXiv preprint arXiv:2306.14921*, 2023.
[17] ——, "ExtractGPT: Exploring the potential of large language models for product attribute value extraction," in *Proc. iiWAS*, ser. LNCS, vol. 15342. Springer, 2024.
[18] A. Brinkmann, R. Baumann, and C. Bizer, "Using LLMs for the extraction and normalization of product attribute values," in *Proc. ADBIS*, ser. LNCS, vol. 14918. Springer, 2024, pp. 217–230.
[19] O. Sainz, I. García-Ferrero, R. Agerri, O. L. de Lacalle, G. Rigau, and E. Agirre, "GoLLIE: Annotation guidelines improve zero-shot information-extraction," in *Proc. ICLR*, 2024.
[20] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, pp. 9459–9474.
[21] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.
[22] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," *Transactions on Machine Learning Research*, 2022.
[23] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
[24] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The Faiss library," *arXiv preprint arXiv:2401.08281*, 2024.
[25] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
[26] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, "Text embeddings by weakly-supervised contrastive pre-training," *arXiv preprint arXiv:2212.03533*, 2022.
[27] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packed resources for general chinese embeddings," in *Proc. ACM SIGIR*, 2024.
[28] Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan, "Deep entity matching with pre-trained language models," *Proc. VLDB Endowment*, vol. 14, no. 1, pp. 50–60, 2020.
[29] A. Narayan, I. Chami, L. Orr, and C. Ré, "Can foundation models wrangle your data?" *Proc. VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022.
[30] R. Peeters and C. Bizer, "Using ChatGPT for entity matching," in *VLDB Workshop on AI & DM*, ser. LNCS. Springer, 2023.
[31] R. Peeters, R. Steiner, and C. Bizer, "Entity matching using large language models," in *Proc. EDBT*, 2025, pp. 529–541.
[32] R. Steiner, R. Peeters, and C. Bizer, "Fine-tuning large language models for entity matching," *arXiv preprint arXiv:2409.08185*, 2024.
[33] Z. Zhang, A. Goswami, D. Bespalov, and Y. Li, "AnyMatch: Efficient zero-shot entity matching with a small language model," *arXiv preprint arXiv:2409.04073*, 2024.
[34] T. Wang, H. Lin, X. Han, L. Sun, and X. Chen, "Match, compare, or select? An investigation of large language models for entity matching," in *Proc. COLING*, 2025, pp. 96–109.
[35] J. L. Fleiss, "Measuring nominal scale agreement among many raters," *Psychological Bulletin*, vol. 76, no. 5, pp. 378–382, 1971.
[36] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

APPENDIX

[Prompts to be included here.]